

Guía Práctica de Python — De Básico a Portafolio

Preparado para Matías González Ferreyra

Agosto 2026

Contents

Introducción	2
Herramientas que vas a necesitar	2
Etapa 1: Fundamentos del lenguaje	2
Teoría	2
Ejemplos	3
Tarea / Proyecto: “Calculadora de datos personales”	3
Etapa 2: Estructuras de control (condicionales y bucles)	3
Teoría	3
Ejemplos	4
Tarea / Proyecto: “Adivina el número”	4
Etapa 3: Estructuras de datos	5
Teoría	5
Ejemplos	5
Tarea / Proyecto: “Agenda de contactos en consola”	5
Etapa 4: Funciones	6
Teoría	6
Ejemplos	6
Tarea / Proyecto: “Calculadora de finanzas personales”	6
Etapa 5: Manejo de archivos y manejo de errores	7
Teoría	7
Ejemplos	7
Tarea / Proyecto: “Registro de gastos personales”	8
Etapa 6: Programación orientada a objetos (básico)	8
Teoría	8
Ejemplos	8
Tarea / Proyecto: “Sistema de inventario simple”	9
Etapa 7: Módulos, entornos virtuales y librerías externas	9
Teoría	9
Ejemplos	10
Tarea / Proyecto: “Análisis de datos con pandas”	10
Proyecto final integrador: “Gestor de Cuentas y Contraseñas” (mejorado)	10
Recursos recomendados	10

Introducción

Esta guía está pensada para que pases de un nivel básico de Python (donde ya conoces variables, tipos de datos y algo de lógica) a un nivel donde puedas escribir programas completos, subirlos a GitHub, y usarlos como portafolio real para buscar empleo en análisis de datos o desarrollo.

Cada etapa tiene tres partes:

1. **Teoría** explicada con palabras simples.
2. **Ejemplos** de código comentados.
3. **Tarea o mini-proyecto** que debes subir a tu GitHub al terminar la etapa.

No avances a la siguiente etapa hasta completar el proyecto de la anterior. La constancia importa más que la velocidad.

Herramientas que vas a necesitar

Herramienta	Para qué sirve	Dónde conseguirla
Python 3.12+	El lenguaje en sí	python.org/downloads
Visual Studio Code	Editor de código	code.visualstudio.com
Extensión "Python" de VS Code	Autocompletado, debugging	Desde el Marketplace de VS Code
Git	Control de versiones	git-scm.com
Cuenta de GitHub	Portafolio público	github.com (ya tienes una)
Terminal (Bash o PowerShell)	Ejecutar tus scripts	Ya viene en tu sistema

Instala Python y marca la opción "Add Python to PATH" durante la instalación. Verifica que todo funciona abriendo una terminal y escribiendo:

```
python --version
git --version
```

Si ambos comandos responden con un número de versión, estás listo.

Etapa 1: Fundamentos del lenguaje

Teoría

Un programa es una lista de instrucciones que la computadora ejecuta en orden, de arriba hacia abajo. Los bloques básicos son:

- **Variable:** una caja con nombre donde guardas un valor. Ejemplo: edad = 25.
- **Tipo de dato:** qué clase de valor guarda una variable.
 - int → números enteros (25)
 - float → números con decimales (25.5)
 - str → texto ("Matías")
 - bool → verdadero o falso (True / False)

- **Operadores:** símbolos que combinan valores. Aritméticos (+ - * / // % **), de comparación (== != > < >= <=), lógicos (and or not).
- **Entrada y salida:** `input()` para pedir datos al usuario, `print()` para mostrar resultados.

Piensa en las variables como etiquetas en cajas de un almacén: la etiqueta (nombre de variable) te dice qué hay dentro (el valor), y puedes cambiar el contenido de la caja cuando quieras.

Ejemplos

```
# Variables y tipos de datos
nombre = "Matías"      # str
edad = 27              # int
altura = 1.75          # float
es_employed = True     # bool

# Operadores aritméticos
suma = 10 + 5
resto = 10 % 3          # el residuo de una división (1)
potencia = 2 ** 3       # 2 elevado a la 3 (8)

# Entrada y salida
nombre_usuario = input("¿Cómo te llamas? ")
print(f"Hola, {nombre_usuario}, bienvenido")

# Comparaciones y lógica
mayor_de_edad = edad >= 18
print(mayor_de_edad)    # True
```

Tarea / Proyecto: “Calculadora de datos personales”

Crea un script `datos_personales.py` que:

1. Pida al usuario su nombre, edad y peso (en kg).
2. Calcule y muestre en qué año cumplirá 100 años.
3. Convierta el peso a libras (1 kg = 2.20462 lb) y lo muestre.
4. Indique con un mensaje si es mayor o menor de edad.

Sube este archivo a un repositorio nuevo en GitHub llamado `python-practica`, con un `README.md` breve explicando qué hace el script.

Etapas 2: Estructuras de control (condicionales y bucles)

Teoría

Los programas casi nunca ejecutan todo en línea recta; necesitan **decidir** y **repetir**.

- **Condicionales (if / elif / else):** le dicen al programa “si se cumple esto, haz A; si no, haz B”. Es como una bifurcación en un camino.
- **Bucle for:** repite un bloque de código un número conocido de veces, o recorre elemento por elemento algo (como una lista o un rango de números).
- **Bucle while:** repite un bloque de código **mientras** una condición sea verdadera. Útil cuando no sabes de antemano cuántas veces se repetirá.

- **break y continue:** break corta el bucle por completo; continue salta a la siguiente vuelta sin terminar la actual.

Un bucle mal controlado (sin una condición que eventualmente sea falsa) causa un **bucle infinito** — el programa nunca termina. Siempre revisa que la condición de un while pueda volverse falsa.

Ejemplos

```
# Condicional
nota = 75
if nota >= 90:
    print("Excelente")
elif nota >= 70:
    print("Aprobado")
else:
    print("Reprobado")

# Bucle for: recorrer un rango de números
for i in range(1, 6):
    print(f"Vuelta número {i}")

# Bucle for: recorrer una lista
frutas = ["manzana", "pera", "uva"]
for fruta in frutas:
    print(fruta)

# Bucle while
intentos = 0
while intentos < 3:
    print("Intento", intentos + 1)
    intentos += 1 # sin esto, el bucle sería infinito

# break y continue
for numero in range(1, 10):
    if numero == 5:
        break # corta el bucle al llegar a 5
    if numero % 2 == 0:
        continue # se salta los números pares
    print(numero)
```

Tarea / Proyecto: “Adivina el número”

Crea adivina_numero.py que:

1. Genere un número aleatorio entre 1 y 100 (usa `import random` y `random.randint(1, 100)`).
2. Pida al usuario que adivine, usando un bucle while que siga preguntando hasta acertar.
3. Después de cada intento, indique si el número secreto es “más alto” o “más bajo”.
4. Al final, muestre cuántos intentos usó.

Súbelo al mismo repositorio python-practica.

Etapa 3: Estructuras de datos

Teoría

Además de guardar un solo valor, Python te deja guardar colecciones de valores:

- **Lista (list):** colección ordenada y modificable. `frutas = ["manzana", "pera"]`
- **Tupla (tuple):** como una lista, pero no se puede modificar después de creada. `coordenadas = (19.4, -99.1)`
- **Diccionario (dict):** pares de clave-valor, como una libreta de contactos donde buscas por nombre. `persona = {"nombre": "Matías", "edad": 27}`
- **Conjunto (set):** colección sin orden y sin elementos repetidos. Útil para eliminar duplicados.

Piensa en una lista como un estante de libros en orden; un diccionario como un directorio telefónico donde buscas por nombre y obtienes un número.

Ejemplos

```
# Listas
tareas = ["comprar leche", "estudiar Python", "hacer ejercicio"]
tareas.append("dormir temprano")    # agregar un elemento
tareas.remove("comprar leche")      # quitar un elemento
print(tareas[0])                    # acceder al primer elemento

# Diccionarios
empleado = {
    "nombre": "Matías",
    "puesto": "Técnico de Campo",
    "activo": True
}
print(empleado["puesto"])
empleado["puesto"] = "Analista de Datos"  # actualizar un valor

# Recorrer un diccionario
for clave, valor in empleado.items():
    print(clave, "->", valor)

# Sets: eliminar duplicados
numeros = [1, 2, 2, 3, 3, 3]
unicos = set(numeros)
print(unicos)    # {1, 2, 3}
```

Tarea / Proyecto: “Agenda de contactos en consola”

Crea `agenda_contactos.py` que use un diccionario donde cada clave sea el nombre de un contacto y el valor sea otro diccionario con teléfono y correo. El programa debe, en un menú con bucle `while`, permitir:

1. Agregar un contacto.
2. Buscar un contacto por nombre.
3. Eliminar un contacto.
4. Listar todos los contactos.
5. Salir del programa.

Este es tu primer proyecto con estructura de menú — muy común en entrevistas técnicas. Súbelo a python-practica.

Etapa 4: Funciones

Teoría

Una función es un bloque de código reutilizable al que le das un nombre. En vez de repetir el mismo código varias veces, lo escribes una vez dentro de una función y la “llamas” cuando la necesitas.

- Se define con `def nombre_funcion(parametros):`
- Puede recibir **parámetros** (datos de entrada) y devolver un resultado con `return`.
- Las variables creadas dentro de una función son locales: no existen fuera de ella (esto se llama **alcance** o *scope*).

Piensa en una función como una receta de cocina: recibe ingredientes (parámetros), sigue un proceso, y entrega un platillo (el `return`). Puedes usar la misma receta las veces que quieras sin reescribirla.

Ejemplos

```
def saludar(nombre):
    return f"Hola, {nombre}"

mensaje = saludar("Matías")
print(mensaje)

# Función con valor por defecto
def calcular_precio(precio, descuento=0):
    return precio - (precio * descuento / 100)

print(calcular_precio(100))          # 100 (sin descuento)
print(calcular_precio(100, 20))     # 80 (20% de descuento)

# Función con varios parámetros y validación
def es_primo(numero):
    if numero < 2:
        return False
    for i in range(2, numero):
        if numero % i == 0:
            return False
    return True

for n in range(1, 20):
    if es_primo(n):
        print(n, "es primo")
```

Tarea / Proyecto: “Calculadora de finanzas personales”

Crea `finanzas.py` con funciones separadas para:

1. `calcular_ahorro_mensual(ingreso, gastos)` — devuelve cuánto se ahorra al mes.
2. `proyectar_ahorro(ahorro_mensual, meses)` — devuelve el total ahorrado tras cierta cantidad de meses.
3. `calcular_interes(monto, tasa_anual, anios)` — calcula interés compuesto simple.

El programa principal debe pedir los datos al usuario, llamar a las funciones, y mostrar un resumen. Este proyecto es un buen ejemplo de “separar lógica en funciones”, algo que se evalúa mucho en entrevistas.

Etapa 5: Manejo de archivos y manejo de errores

Teoría

Hasta ahora tus programas pierden toda la información al cerrarse. Para guardar datos de forma permanente, necesitas leer y escribir archivos.

- `open("archivo.txt", "r")` → abre un archivo para **leer** (`r` = read).
- `open("archivo.txt", "w")` → abre un archivo para **escribir**, borrando lo anterior (`w` = write).
- `open("archivo.txt", "a")` → abre un archivo para **agregar** contenido sin borrar (`a` = append).
- Se recomienda usar `with open(...) as archivo:` porque cierra el archivo automáticamente al terminar.

Además, los programas reales fallan (un archivo no existe, el usuario escribe texto donde se esperaba un número). Para eso existe el manejo de errores:

- `try:` — el bloque de código que puede fallar.
- `except:` — qué hacer si falla, en vez de que el programa se cierre de golpe.

Piensa en `try/except` como una red de seguridad bajo un trapecista: si algo sale mal, no te caes al vacío, tienes un plan B.

Ejemplos

```
# Escribir en un archivo
with open("notas.txt", "w") as archivo:
    archivo.write("Primera nota\n")
    archivo.write("Segunda nota\n")

# Leer un archivo línea por línea
with open("notas.txt", "r") as archivo:
    for linea in archivo:
        print(linea.strip())

# Manejo de errores
try:
    numero = int(input("Escribe un número: "))
    resultado = 10 / numero
    print(resultado)
except ValueError:
    print("Eso no es un número válido")
except ZeroDivisionError:
```

```

    print("No se puede dividir entre cero")

# Trabajar con archivos CSV (tablas de datos)
import csv

with open("gastos.csv", "w", newline="") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(["fecha", "categoria", "monto"])
    escritor.writerow(["2026-08-01", "comida", 250])

with open("gastos.csv", "r") as archivo:
    lector = csv.reader(archivo)
    for fila in lector:
        print(fila)

```

Tarea / Proyecto: “Registro de gastos personales”

Crea `registro_gastos.py` que:

1. Guarde gastos (fecha, categoría, monto) en un archivo CSV.
2. Permita agregar un gasto nuevo desde un menú.
3. Permita mostrar el total gastado por categoría (usando un diccionario para acumular).
4. Maneje errores si el usuario escribe un monto que no es un número.

Este proyecto es muy relevante para tu meta de análisis de datos: ya estás manipulando datos tabulares con Python.

Etapa 6: Programación orientada a objetos (básico)

Teoría

Hasta ahora tus programas usan funciones sueltas. La programación orientada a objetos (POO) organiza el código en **clases**, que son “moldes” para crear objetos con datos y comportamientos propios.

- **Clase:** el molde o plantilla (ej. Empleado).
- **Objeto:** una instancia concreta de la clase (ej. el empleado “Matías”).
- **Atributos:** los datos que guarda el objeto (nombre, puesto).
- **Métodos:** las funciones que pertenecen a la clase (lo que el objeto puede “hacer”).
- `__init__` es un método especial que se ejecuta automáticamente al crear un objeto nuevo, para inicializar sus datos.

Piensa en una clase como el plano de una casa, y en cada objeto como una casa concreta construida con ese plano — todas comparten la misma estructura, pero cada una tiene sus propios muebles (datos).

Ejemplos

```

class Empleado:
    def __init__(self, nombre, puesto, salario):
        self.nombre = nombre
        self.puesto = puesto

```



```

        self.salario = salario

    def dar_aumento(self, porcentaje):
        self.salario += self.salario * (porcentaje / 100)

    def mostrar_info(self):
        print(f"{self.nombre} - {self.puesto} - ${self.salario:.2f}")

# Crear objetos (instancias) de la clase
empleado1 = Empleado("Matías", "Técnico de Campo", 15000)
empleado2 = Empleado("Ana", "Analista", 18000)

empleado1.dar_aumento(10)
empleado1.mostrar_info()
empleado2.mostrar_info()

```

Tarea / Proyecto: “Sistema de inventario simple”

Crea inventario.py con una clase Producto (nombre, precio, cantidad) que tenga métodos para:

1. Vender unidades (restar del inventario, validando que no se venda más de lo disponible).
2. Reabastecer inventario.
3. Calcular el valor total del inventario (precio × cantidad).

El programa principal debe crear varios productos y simular ventas y reabastecimientos desde un menú.

Etapa 7: Módulos, entornos virtuales y librerías externas

Teoría

Python trae muchas herramientas incluidas (“librería estándar”: random, csv, os, datetime), pero el verdadero poder viene de librerías externas que otras personas crearon, como **pandas** (análisis de datos) o **requests** (conectarse a internet).

- `pip install nombre_libreria` instala una librería.
- Un **entorno virtual** (venv) es una carpeta aislada donde instalas librerías solo para un proyecto, sin mezclarlas con otros. Esto evita conflictos de versiones entre proyectos distintos.

```

python -m venv entorno
# En Windows:
entorno\Scripts\activate
# En Mac/Linux:
source entorno/bin/activate

pip install pandas

```

Ejemplos

```
import pandas as pd

datos = {
    "producto": ["Aceite", "Grasa", "Refrigerante"],
    "cantidad": [50, 30, 20],
    "precio": [120.5, 200.0, 95.75]
}

df = pd.DataFrame(datos)
print(df)

df["total"] = df["cantidad"] * df["precio"]
print(df.sort_values("total", ascending=False))

df.to_csv("inventario_quimicos.csv", index=False)
```

Tarea / Proyecto: “Análisis de datos con pandas”

Toma el archivo CSV que generaste en la Etapa 5 (registro de gastos) y crea `analis_gastos.py` que, usando pandas:

1. Cargue el CSV en un DataFrame.
2. Calcule el gasto total, el promedio por categoría, y la categoría con más gasto.
3. Exporte un resumen a un nuevo archivo CSV.

Este proyecto conecta directamente tu experiencia real en Excel (Quaker Houghton) con Python — es un excelente punto para tu portafolio de análisis de datos.

Proyecto final integrador: “Gestor de Cuentas y Contraseñas” (mejorado)

Ya mencionas este proyecto en tu CV. Con lo aprendido en esta guía, mejóralo para que tenga:

1. Clases (Usuario, Credencial) en vez de solo funciones sueltas.
2. Guardado persistente en un archivo (CSV o JSON, usando `import json`).
3. Manejo de errores robusto (contraseñas duplicadas, usuario no encontrado).
4. Menú de consola claro con opciones numeradas.
5. Un README.md completo: qué hace el proyecto, cómo instalarlo, cómo usarlo, capturas de pantalla si es posible.

Súbelo a GitHub como proyecto destacado (fíjalo/*pin* en tu perfil). Este es el proyecto que debes enlazar en tu CV con una descripción específica, no genérica.

Recursos recomendados

- **Documentación oficial:** docs.python.org/es/3/ (en español).
- **Práctica de ejercicios:** exercism.org (Python track), [HackerRank](https://www.hackerrank.com) (Python domain).

- **Curso que ya estás tomando:** termina el especializado de Coursera antes de avanzar a librerías más avanzadas como NumPy o Matplotlib.
- **Siguiente paso natural después de esta guía:** aprender **NumPy** y **Matplotlib/Seaborn** para visualización de datos, ya que tu meta es análisis de datos.

Cómo subir cada proyecto a GitHub

```
cd carpeta_del_proyecto
git init
git add .
git commit -m "Primer commit: agrego proyecto X"
git branch -M main
git remote add origin https://github.com/MatiasGonzalez-Dev/python-practica.git
git push -u origin main
```

Para los proyectos siguientes dentro del mismo repositorio, basta con `git add .`, `git commit -m "mensaje"` y `git push`.